SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Presentado por:

Sebastian Cifuentes Moncada

Josue Penagos Amaya

09/10/2026

Inteligencia de negocios (BI)

Institucion Universitaria Pascual Bravo

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Estrategia de Articulación EnContexto · Departamento de Sistemas Digitales

Trabajo de consulta en parejas

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Inteligencia de Negocios · Grupo 01 · Semestre 2026-II · Docente: Ramiro Grisales Montoya

1. Propósito

En SolarBI el cuadro de mando final se construye en Power BI. Sin embargo, en la industria el monitoreo de plantas y equipos suele hacerse con Grafana. En esta consulta compararán las dos herramientas sobre los mismos datos y darán sus primeros pasos en tres temas que sostienen el proyecto: gobernanza de datos, automatización de procesos ETL e Internet de las Cosas (IoT), trabajando con control de versiones en GitHub.

> **Enfoque del Grupo 01:** monitoreo operativo. Su práctica se centra en la potencia en el tiempo y en la detección de fallas.

2. Ficha del trabajo

Campo	Contenido
Curso / Grupo	Inteligencia de Negocios · Grupo 01
Modalidad	Parejas (dos integrantes). Una sola entrega por pareja.
Tiempo disponible	Dos (2) días a partir de la publicación en Google Classroom.
Fecha límite	
Valor	
Articulación	Proyecto de Aula SolarBI Pascual. Prepara el Entregable III (esquema estrella y flujo ETL, semana 12) y el Entregable V (cuadro de mando, semana 16).
Entrega	Un archivo PDF central + repositorio en GitHub con los archivos de soporte.

Indicadores de resultado de aprendizaje que moviliza: U2-a (lago de datos), U2-b (calidad de datos), U2-d (flujos ETL), U3-d (indicadores de desempeño), U3-e (cuadros de mando) y U2-e (responsabilidad frente a la información).

3. Parte A · Consulta

Respondan cada pregunta con sus propias palabras, en máximo 120 palabras, aplicándola al caso SolarBI e indicando la fuente. Las tablas, los diagramas y los fragmentos de código no cuentan en el límite.

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana? Describan sus componentes: data sources, panels, dashboards, variables y alerting. Diferencien Grafana OSS de Grafana Cloud.

Grafana es una plataforma de código abierto para visualizar y monitorear datos provenientes de diferentes fuentes.

Componentes (Grafana Labs, s. f.-d) :

- **Data Sources:** Son las fuentes donde Grafana obtiene los datos, como PostgreSQL, MySQL, InfluxDB o Prometheus (Grafana Labs, s. f.-c).
- **Panels**: Son los elementos individuales que muestran información, por ejemplo gráficos, tablas, indicadores o mapas (Grafana Labs, s. f.-f) .
- **Dashboards:** Son conjuntos de panels organizados en una misma pantalla (Grafana Labs, s. f.-b). .
- **Variables:** son las que permiten cambiar dinámicamente valores del dashboard, como un sitio dispositivo o periodo (Grafana Labs, s. f.-g) .
- **Alerting:** Permite crear reglas que detectan determinadas condiciones y generan notificaciones (Grafana Labs, s. f.-a).

La diferencia que hay entre Grafana OSS y Grafana Cloud es que Grafana OSS es la versión de código abierto que se instala y administra localmente, mientras que Grafana Cloud es un servicio administrado en la nube que incluye Grafana y servicios adicionales sin tener que administrar directamente la infraestructura (Grafana Labs, s. f.-e).

D2. Elaboren una tabla comparativa Power BI vs. Grafana con mínimo 8 criterios: propósito principal, usuario típico, conexión a los datos, modelo semántico y lenguaje (DAX frente a la consulta del origen), actualización y tiempo real, alertas, licenciamiento y costo, y control de acceso.

Power BI	Grafana
Propósito principal: Power BI está principalmente enfocado en inteligencia de negocios ayudando a responder preguntas estratégicas como: "¿cuáles fueron las ventas del trimestre?" o "¿qué producto es más rentable?" (Microsoft, s. f.-f)	**Propósito principal:** Grafana está más enfocado en observabilidad y monitoreo, ayudando a responder preguntas operativas inmediatas como: "¿está caído el servidor?", "¿cuánta memoria RAM consume el clúster de Kubernetes?" o "¿cuál es la tasa de errores de la API en los últimos 10 segundos?". (Grafana Labs, s. f.-d)
Usuario típico: Los usuarios principales de Power BI son analistas de datos, directivos, gerentes, contadores y equipos comerciales o financieros.	**Usuario típico:** Los usuarios de Grafana son ingenieros DevOps, administradores de sistemas (SysAdmins), ingenieros de confiabilidad del sitio (SRE) y desarrolladores de software
Conexión a los datos: Power BI se conecta principalmente a aplicaciones corporativas, bases de datos relacionales y archivos de gestión empresarial (como SQL Server, Excel, SAP, Salesforce, Azure Synapse y BigQuery) (Microsoft, s. f.-d)	**Conexión a los datos:** Grafana se conecta predominantemente a bases de datos de series temporales, gestores de eventos y plataformas de infraestructura (como Prometheus, InfluxDB, Loki, Tempo, Elasticsearch, CloudWatch y OpenTelemetry). (Grafana Labs, s. f.-b)
Modelo semántico y lenguaje: Power BI se conecta principalmente a aplicaciones corporativas, bases de datos relacionales y archivos de gestión empresarial (como SQL Server, Excel, SAP, Salesforce, Azure Synapse y BigQuery)	**Modelo semántico y lenguaje:** No tiene :cada panel envía una consulta en el lenguaje del origen (SQL en PostgreSQL, PromQL en Prometheus) y solo permite transformaciones simples sobre el resultado (Grafana Labs, s. f.-h).
Actualización y tiempo real: En modo importación los datos se actualizan según un horario programado; con DirectQuery se consulta el origen al abrir el informe. El tiempo real existe (streaming), pero es más limitado (Microsoft, s. f.-a; s. f.-j).	**Actualización y tiempo real:** Cada panel consulta el origen directamente y el dashboard se puede refrescar automáticamente cada pocos segundos, por lo que está pensado para monitoreo casi en tiempo real (Grafana Labs, s. f.-a).
Alertas: Las alertas de datos se crean sobre tiles de un dashboard publicado (tarjetas, KPI, medidores) y solo se evalúan cuando los datos se actualizan (Microsoft, s. f.-b).	**Alertas:** Grafana Alerting usa alert rules (consulta, condición, intervalo) que se evalúan continuamente y notifican por contact points como correo, Telegram, Slack o webhook (Grafana Labs, s. f.-c).
Licenciamiento y costo: Es un producto comercial de Microsoft: Power BI Desktop es gratuito, pero publicar y compartir informes requiere licencias de pago por usuario o por capacidad (Microsoft, s. f.-e).	**Licenciamiento y costo:** Es un producto comercial de Microsoft: Power BI Desktop es gratuito, pero publicar y compartir informes requiere licencias de pago por usuario o por capacidad (Microsoft, s. f.-e).

Power BI	Grafana
Control de acceso: Usa permisos por workspace y seguridad a nivel de fila (RLS) con roles y filtros DAX para limitar qué filas ve cada usuario (Microsoft, s. f.-i).	**Control de acceso:** Usa organizaciones, equipos y permisos por carpeta o dashboard (Viewer, Editor, Admin); no tiene RLS nativo (Grafana Labs, s. f.-i).

D3. Diferencien un dashboard operativo, uno analítico y uno estratégico. Clasifiquen cada KPI mínimo del proyecto (energía, yield y Performance Ratio, ahorro, disponibilidad y alarmas, porcentaje de datos válidos) e indiquen en cuál de las dos herramientas lo publicarían y por qué.

Un dashboard operativo muestra el estado actual y facilita reaccionar ante problemas. Un dashboard analítico permite estudiar tendencias y comparar resultados. Un dashboard estratégico resume indicadores importantes para apoyar decisiones de alto nivel (Few, 2013).

KPI	Tipo	Herramienta	¿Por que?
Energía (kWh)	Operativo	Grafana	Se monitorea de forma frecuente (cada 5 minutos) y permite ver de inmediato si la planta produce lo esperado. Grafana consulta PostgreSQL y se refresca en segundos (Grafana Labs, s. f.-a).
Yield (kWh / kWp)	Analitico	Power BI	Es la energía producida por cada kWp instalado (IEC, 2021). Sirve para comparar días, meses y sitios, y eso se hace mejor con un modelo semántico y medidas DAX (Microsoft, s. f.-h).
Performance Ratio	Analitico	Power BI	Relaciona el yield final con el de referencia según la irradiancia y evalúa el rendimiento real de la planta (IEC, 2021). Interesa su evolución histórica, no el valor puntual.
Ahorro	Estratégico	Power BI	Resume el resultado económico para quienes toman decisiones. Se actualiza por horario y se presenta con indicadores y tarjetas de alto nivel (Microsoft, s. f.-a).
Disponibilidad	Operativo	Grafana	Detecta rápido si un equipo dejó de reportar o está fuera de servicio, y puede disparar una alerta continua (Grafana Labs, s. f.-c).

Alarmas	Operativo	Grafana	Requieren atención inmediata. Las alert rules se evalúan continuamente y notifican por correo, Telegram o Slack (Grafana Labs, s. f.-c).
% Datos validos	Operativo	Grafana	Controla la calidad de los datos casi en tiempo real: si baja del umbral (por ejemplo 95 %), se avisa antes de que afecte los reportes.

D4.Grafana con PostgreSQL: ¿cómo debe ser una consulta de series de tiempo y para qué sirven las macros `$__timeFilter` y `$__timeGroup`? Escriban la consulta de la potencia promedio por hora.

Una consulta de series de tiempo en Grafana con PostgreSQL debe devolver una columna de fecha/hora (llamada time o con alias), una o más columnas numéricas con la medida, y estar ordenada por tiempo (Grafana Labs, s. f.-h) . La macro $__timeFilter(columna) genera el WHERE con el rango seleccionado en el dashboard, así solo se leen los datos visibles. $__timeGroup(columna, ‘1h’) agrupa las lecturas en intervalos fijos para poder promediar. En SolarBI, así se obtiene la potencia promedia por hora del inversor sin traer los miles de lecturas de 5 minutos

```
SELECT
  $__timeGroup(ts, '1h') AS time,
  AVG(p_ac_kw)           AS potencia_prom_kw
FROM silver.lectura_5min
WHERE $__timeFilter(ts)
GROUP BY 1
ORDER BY 1;
```

D5.Alertas: comparen Grafana Alerting (alert rules y contact points) con las alertas de datos del servicio de Power BI. Propongan una regla de alerta útil para la planta y el canal por el que se notificará.

En Grafana Alerting se define una alert rule (consulta + condición + intervalo de evaluación + tiempo de espera "pending") que se evalúa de forma continua. Si se cumple, una política de notificación la envía a un contact point (correo, Telegram, Slack, Teams o webhook) (Grafana Labs, s. f.-c). En Power BI las alertas de datos se crean sobre tiles de un dashboard publicado (tarjetas, KPI, medidores) y solo se evalúan cuando los datos se actualizan, por lo que son más limitadas y no sirven para vigilancia continua (Microsoft, s. f.-b).

Bloque 2 · Gobernanza de datos (data governance)

G1.Definan data governance y diferenciarla de data management y de data quality. Tomen como referencia el marco DAMA-DMBOK.

Según DAMA-DMBOK, data governance es el ejercicio de autoridad y control sobre los activos de datos: define políticas, roles, responsabilidades y decisiones sobre quién puede hacer qué con los datos . Data management es más amplio: es el conjunto de prácticas que ejecutan esas políticas (arquitectura, almacenamiento, integración, seguridad, metadatos) . Data quality es una de esas áreas y mide si los datos sirven para su uso (exactitud, completitud, consistencia) (DAMA International, 2017).

G2.Expliquen los roles data owner, data steward y data custodian. Relaciónalos con los roles del equipo en el proyecto (data engineer, data modeler, BI analyst y product owner).

El data owner es quien responde por un conjunto de datos y decide su uso y acceso. El data steward gestiona el día a día: define significados, reglas de calidad y corrige problemas (DAMA International, 2017). El data custodian se encarga de la parte técnica: almacenamiento, copias de seguridad y seguridad. En el proyecto: el product owner actúa como data owner (decide qué se mide y para qué); el data modeler y el BI analyst son stewards (definen KPIs, dimensiones y reglas de calidad); y el data engineer es el custodian (opera PostgreSQL, el ETL y los accesos).

G3.El contrato de datos (data contract) del proyecto como instrumento de gobernanza: ¿qué debe incluir (esquema, unidades, frecuencia, responsable, niveles de calidad)? ¿Qué le pasa al cuadro de mando si la fuente cambia un campo sin avisar?

Un contrato de datos es un acuerdo formal entre quien produce los datos y quien los consume, que describe qué datos se entregan y en qué condiciones (Jones, 2023; Bitol, s. f.). Como instrumento de gobernanza, debe incluir el esquema (campos y tipos), las unidades (kW, W/m², °C), la frecuencia (cada 5 min) y la zona horaria, el responsable de la fuente, los niveles de calidad (por ejemplo, ≥ 95 % de datos válidos y sin duplicados), la versión y la política de cambios (Bitol, s. f.).

G4.Seguridad a nivel de fila (row-level security, RLS) en Power BI: roles y filtros DAX. Den el ejemplo de un responsable que solo ve su sitio. ¿Cuál es el mecanismo equivalente de control de acceso en Grafana (organizaciones, equipos y permisos por carpeta o dashboard)?

La seguridad a nivel de fila (RLS) en Power BI restringe qué filas ve cada usuario según su rol. En Power BI Desktop se crea el rol (Modelado → Administrar roles) y se le asigna un filtro DAX sobre una tabla. Después, en el servicio, se asignan los usuarios o grupos al rol (Microsoft, s. f.-i).

Ejemplo: en la tabla dim_sitio se define el filtro [sitio_id] = "MED01". Así el responsable de Medellín solo ve su sitio, y el filtro se propaga a fact_energia_dia a través de las relaciones del modelo. También se puede usar un rol dinámico con el filtro [correo_responsable] = USERPRINCIPALNAME(), que compara el correo de cada usuario con el de la tabla y evita crear un rol por sitio (Microsoft, s. f.-i).

Grafana no tiene RLS por fila. Controla el acceso con organizaciones, equipos y permisos por carpeta o dashboard (Viewer, Editor, Admin) (Grafana Labs, s. f.-j). Para limitar los datos por sitio se usa un dashboard por sitio, o un usuario de PostgreSQL con permisos restringidos en el data source.

G5.Ley 1581 de 2012 (protección de datos personales en Colombia): ¿los datos de SolarBI son personales? ¿Qué cambiaría si el tablero mostrará qué operador atendió cada alarma? Fijen la postura del equipo frente al manejo responsable de la información.

Según la Ley 1581 de 2012, un dato personal es cualquier información vinculada o que pueda asociarse a una o varias personas naturales determinadas o determinables (Congreso de Colombia, 2012, art. 3). La telemetría de SolarBI (potencia, irradiancia, temperatura del módulo) describe un equipo y no a una persona, así que no es dato personal.

Si el tablero mostrará qué operador atendió cada alarma, sí lo sería, porque identificaría a alguien y permitiría evaluar su desempeño. Entonces habría que definir una finalidad clara y legítima, pedir autorización previa, expresa e informada del titular, y restringir el acceso a los datos (Congreso de Colombia, 2012, arts. 4 y 9).

La postura del equipo sería recolectar solo los datos necesarios (un código de operador en lugar del nombre), limitar quién los ve con RLS, no subir datos reales ni credenciales al repositorio y dejar la finalidad escrita en el contrato de datos.

Bloque 3 · Automatización de procesos ETL

E1.Diferencien ETL de ELT y ubiquen cada paso en las capas Bronze, Silver y Gold del proyecto. ¿Power Query es una herramienta ETL? Argumenten.

ETL extrae, transforma y luego carga: la limpieza ocurre en un motor intermedio antes del destino. ELT extrae, carga primero el dato crudo y lo transforma dentro del destino, aprovechando su capacidad de cómputo (Amazon Web Services, s. f.).

En SolarBI (arquitectura medallion; Databricks, s. f.), Bronze es el CSV crudo sin modificar; Silver son las lecturas limpias y validadas, cargadas en silver.lectura_5min; y Gold es el resumen diario en fact_energia_dia, calculado con SQL dentro de PostgreSQL. Es un flujo mixto se usa ETL hasta Silver (se limpia antes de cargar) y ELT en Gold.

Power Query sí es una herramienta ETL de autoservicio: extrae de muchas fuentes, transforma con pasos aplicados y carga al modelo (Microsoft, s. f.-g).

E2.Comparen carga completa (full load) y carga incremental. ¿Qué significa que una carga sea idempotente y cómo se logra en `fact_energia_dia` con su clave primaria (`fecha_key`, `dispositivo_key`) y un UPSERT?

La carga completa borra y recarga todo el destino en cada ejecución; es simple pero costosa ya que si crece el volumen. La incremental procesa solo lo nuevo o modificado desde la última carga. Una carga es importante si se ejecuta una vez o diez bajo el mismo estado final, sin filas duplicadas. En fact_energia_dia la clave primaria identifica de forma única cada día y dispositivo. El UPSERT inserta la fila si no existe y la actualiza si

ya existe (PostgreSQL, Global Development Group s.f). Asi, correr run_elt.py dos veces mantiene los mismos conteos

```
INSERT INTO dwh.fact_energia_dia (fecha_key, dispositivo_key, fecha,energia_kwh,
pct_datos_validos) VALUES(:fecha_key, :dispositivo_key, :fecha, :energia_kwh, :pct_datos_validos)
ON CONFLICT (fecha_key, dispositivo_key) do update set energia_kwh = excluded.energia_kwh,
pct_datos_validos = excluded.pct_datos_validos;
```

E3. **Elaboren una tabla comparativa de tres formas de automatizar un flujo: cron (o el Programador de tareas de Windows), Apache Airflow y la actualización programada de Power BI. Incluyan qué es, qué automatiza cada una y cuándo conviene.**

	Programas de tareas	Apache Airflow	Actualización programa de Power BI
Que es	Planificador del sistema operativo que ejecuta comandos en horarios definidos (Linux Programmer's Manual, s. f.). En Windows, equivale al Programador de tareas.	Plataforma de código abierto para crear, programar y monitorear flujos de trabajo (DAGs) escritos en Python (Apache Software Foundation, s. f.).	Función del servicio Power BI que vuelve a consultar los orígenes y actualiza el modelo importado (Microsoft, s. f.-a).
Que Automatiza	Un comando o script a una hora fija, como python etl/run_etl.py cada medianoche.	Varias tareas con dependencias, reintentos, alertas, historial y reprocesos de fechas pasadas.	Solo el refresco del modelo publicado. No limpia datos ni los carga en PostgreSQL.
Cuando conviene	Un script simple y único, como el ETL de SolarBI en esta fase.	Muchos pasos, varias fuentes o necesidad de trazabilidad y recuperación ante fallos.	Cuando el informe ya está publicado y las tablas de PostgreSQL ya están actualizadas.

E4. **Query folding en Power Query: ¿qué es, por qué mejora el rendimiento y cómo se verifica si un paso se está plegando hacia PostgreSQL?**

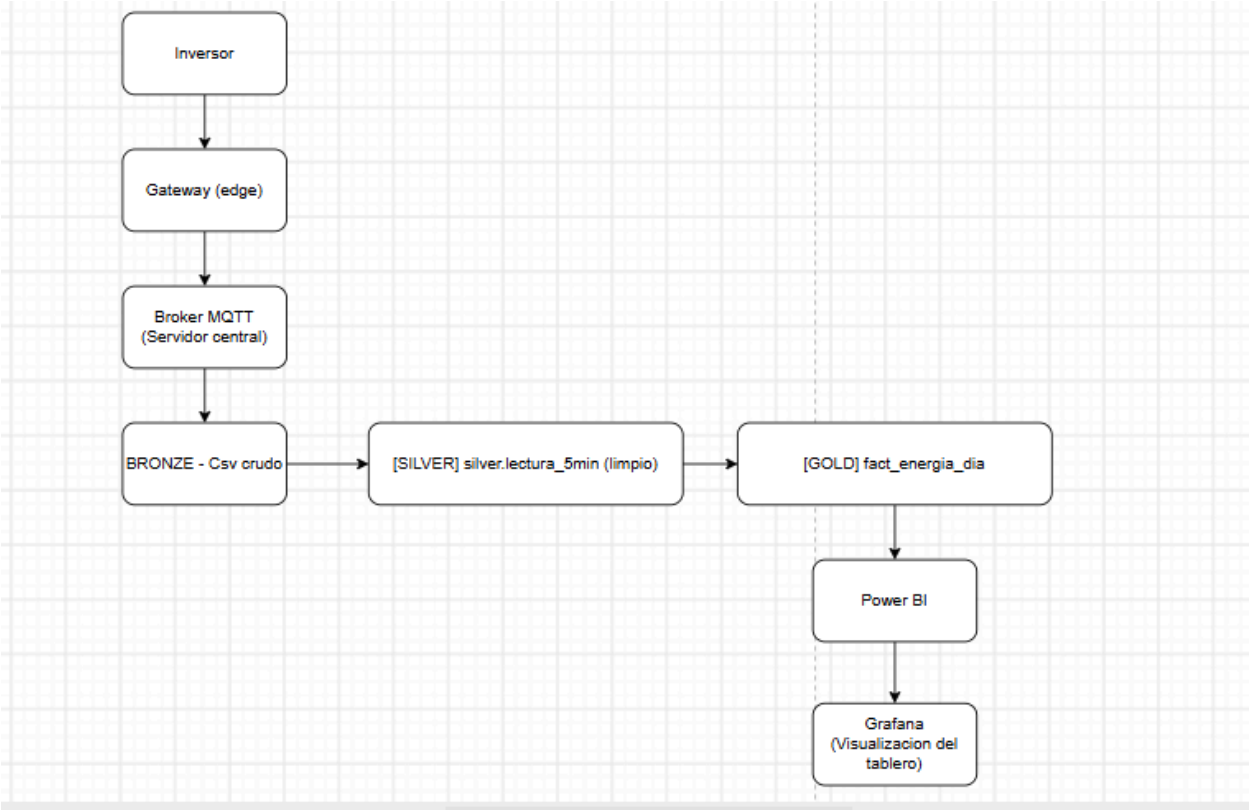
El query folding es la capacidad de Power Query de traducir los pasos de transformación (filtros, agrupaciones, selección de columnas) a una sola consulta SQL que ejecuta el propio origen (Microsoft, s. f.-c). Mejora el rendimiento porque PostgreSQL filtra y agrega con sus índices, y a Power BI solo llega el resultado, no

miles de lecturas de 5 minutos. Para verificar si un paso se pliega, se hace clic derecho sobre él en Pasos aplicados y se revisa "Ver consulta nativa": si está habilitada, el paso se pliega hacia PostgreSQL; si aparece en gris, no. Un paso que rompe el folding impide que los siguientes se plieguen, por eso conviene poner los filtros al inicio.

Bloque 4 · Internet de las Cosas (IoT)

I1. Describan la arquitectura IoT por capas: dispositivo, gateway (edge), red o broker, almacenamiento y aplicación. Dibujen cómo viaja una lectura del inversor hasta llegar a un visual del tablero, indicando en qué capa (Bronze, Silver, Gold) queda en cada momento.

La arquitectura IoT se organiza en capas (IBM, s. f.). En SolarBI, el dispositivo es el inversor con sus sensores (potencia, irradiancia, temperatura). El gateway (edge) reúne esas lecturas y las envía. La red o broker transporta los mensajes por MQTT. El almacenamiento guarda los datos (CSV y PostgreSQL) y la aplicación los muestra en Power BI y Grafana. En la práctica, simulador.py reemplaza al dispositivo y al gateway.



I2. Expliquen MQTT: modelo publish/subscribe, broker, topic y niveles de QoS (0, 1 y 2). Propongan la jerarquía de topics de la planta, por ejemplo `pascualbravo/solar/<sitio>/<dispositivo>/<variable>`.

MQTT es un protocolo de mensajería liviano con modelo publish/subscribe: los dispositivos publican mensajes en un topic (cadena de texto para filtrar y enrutar los mensajes) y los clientes interesados se suscriben, sin conocerse entre sí. El broker recibe los mensajes y los reparte (OASIS, 2019). El QoS define la garantía de entrega: 0 es como máximo una vez, sin confirmación; 1 es al menos una vez, y puede duplicar; 2 es

exactamente una vez. Para la telemetría de 5 minutos conviene QoS 1, y los posibles duplicados se controlan con las reglas de Silver y el UPSERT.

Jerarquía propuesta:

```
pascualbravo/solar/<sitio>/<dispositivo>/<variable>

pascualbravo/solar/med01/inv01/p_ac_kw

pascualbravo/solar/med01/inv01/irradiancia_wm2

pascualbravo/solar/med01/inv01/temp_modulo_c
```

I3. Describan una pila abierta típica de monitoreo IoT: Mosquitto, Telegraf, una base de series de tiempo (InfluxDB o TimescaleDB) y Grafana. ¿Qué papel cumple cada componente y dónde encajaría PostgreSQL?

Una pila abierta típica funciona así: Mosquitto es el broker MQTT que recibe las lecturas; Telegraf se suscribe al broker y las escribe en la base; InfluxDB o TimescaleDB guardan las series de tiempo con su marca de fecha; y Grafana las consulta y las muestra (Eclipse Foundation, s. f.; InfluxData, s. f.-a; s. f.-b; Timescale, s. f.). PostgreSQL encajaría como almacenamiento relacional: TimescaleDB es una extensión de PostgreSQL, y allí viven las tablas Silver y Gold, de donde leen Power BI y Grafana.

I4. Procesamiento por lotes (batch) frente a flujo continuo (streaming). La telemetría llega cada 5 minutos y `fact_energia_dia` tiene granularidad diaria: ¿qué se gana y qué se pierde al agregar? ¿Qué herramienta atiende mejor cada granularidad?

El procesamiento por lotes trabaja sobre datos acumulados, y el flujo continuo procesa cada lectura al llegar (Kleppmann, 2017). Al agregar las lecturas de 5 minutos en un día se gana: menos filas, consultas más rápidas y cifras estables para análisis. Se pierde el detalle, porque ya no se ve la caída de potencia de una hora ni una falla puntual. Grafana atiende mejor los 5 minutos (monitoreo y alertas), y Power BI atiende mejor la granularidad diaria (análisis y reportes).

Bloque 5 · Control de versiones

V1. Diferencien Git de GitHub y definan repository, commit, branch y pull request. Un archivo `.pbix` es binario: ¿qué problema genera en Git y qué resuelve el formato Power BI Project (`.pbip`)? ¿Cómo se versiona un dashboard de Grafana?

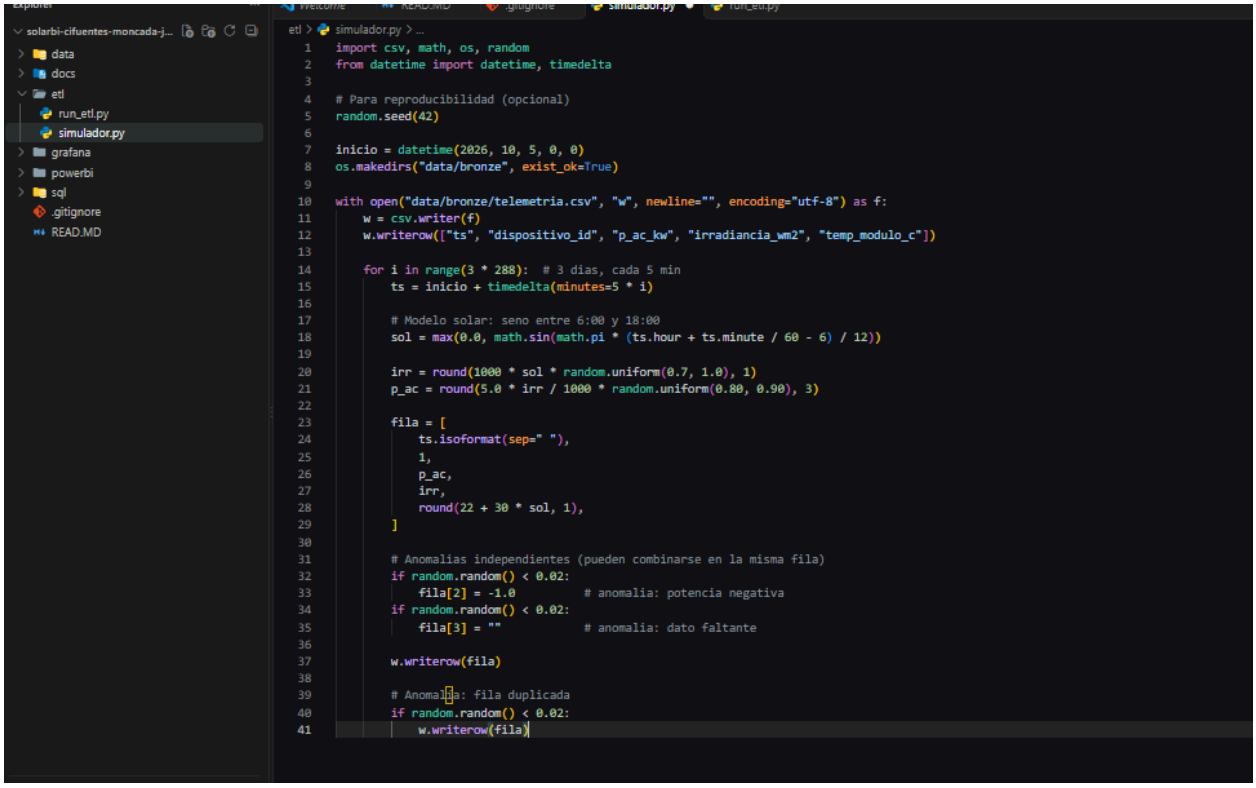
Git es el sistema que registra localmente los cambios del código; GitHub es la plataforma en la nube que aloja repositorios Git y facilita el trabajo en equipo (GitHub, s. f.). Un repository es la carpeta del proyecto con su historial; un commit, un registro de cambios con mensaje; un branch, una línea paralela de desarrollo; y un pull request, la propuesta de fusionar una rama para que el equipo la revise. El .pbix es binario: Git no muestra qué cambió y no puede fusionar conflictos. El .pbip guarda el proyecto como archivos de texto, comparables y versionables (Microsoft, s. f.-f). El dashboard de Grafana se exporta como JSON y se guarda en el repositorio (Grafana Labs, s. f.-l).

4. Parte B · Práctica guiada: los mismos datos en dos herramientas

Construirán un flujo pequeño pero completo: datos crudos de un dispositivo simulado, limpieza con reglas de calidad, carga en PostgreSQL y dos visualizaciones sobre esa misma base, una en Power BI y otra en Grafana.

Paso 1 · Bronze: datos crudos

Dataset:



Paso 2 · Silver: reglas de calidad

Regla 1 - Duplicados:

Se marca como duplicada toda lectura que repite la misma marca de tiempo (ts) y el mismo dispositivo_id, conservando la primera aparición (keep="first"). Esta regla atrapa la anomalía intencional del simulador, que con probabilidad del 2 % escribe dos veces la misma fila. Es clave porque un duplicado inflaría la energía diaria en fact_energia_dia y rompería la idempotencia de la carga.

```
# Regla 1 · Duplicados: misma lectura (ts + dispositivo) repetida. Se conserva la primera.
duplicada = df.duplicated(subset=["ts", "dispositivo_id"], keep="first")
df.loc[duplicada, "motivo"] = "duplicado"
```

Regla 2 - Datos faltantes:

Se marca la fila cuando la fecha (ts) o cualquiera de las tres medidas (p_ac_kw, irradiancia_wm2, temp_modulo_c) está vacía. El simulador introduce esta anomalía escribiendo "" en irradiancia_wm2 con un 2 % de probabilidad. La condición & df["motivo"].isna() garantiza que no se sobreescriba un motivo ya asignado por la regla anterior (el orden importa).

```
# Regla 2 · Datos faltantes: fecha o alguna medida vacía.
faltante = df[["ts"] + COLUMNAS_MEDIDA].isna().any(axis=1) & df["motivo"].isna()
df.loc[faltante, "motivo"] = "dato_faltante"
```

Regla 3 - Rango físico válido:

Al final se separan las filas válidas (motivo nulo) de las rechazadas, y se calcula el porcentaje de datos válidos como 100 * válidas / leídas. Este indicador es el mismo que alimenta la columna pct_datos_validos de dwh.fact_energia_dia y el KPI "% Datos válidos" del tablero (con umbral del 95 %).

```
# Regla 3 · Rango físico válido.
fuera = pd.Series(False, index=df.index)
for columna, (minimo, maximo) in RANGOS.items():
    fuera |= (df[columna] < minimo) | (df[columna] > maximo)
fuera &= df["motivo"].isna()
df.loc[fuera, "motivo"] = "fuera_de_rango"
```

Paso 3 · Gold: carga en PostgreSQL

- **Carguen las lecturas limpias de 5 minutos en una tabla `silver.lectura_5min`.**

  Las lecturas limpias de 5 minutos que produce etl/silver.py (data/silver/telemetria_limpia.csv) se cargan en PostgreSQL mediante el módulo etl/load_postgres.py, que ejecuta un UPSERT (INSERT ... ON CONFLICT (ts, dispositivo_id) DO UPDATE). La clave primaria compuesta (ts, dispositivo_id) definida en sql/01_schema.sql garantiza que una misma lectura no se duplique: si ya existe, se actualizan sus medidas. Las credenciales se leen desde un archivo .env excluido del repositorio mediante .gitignore, y el script se ejecuta desde la raíz del proyecto con el comando:

```
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya> python -m etl.load_silver
Leyendo CSV limpio...
Filas en el CSV: 833

Filas enviadas en esta ejecución: 833
Total en silver.lectura_5min    : 833
```

Modulo load_postgres.py:

```python
# ------------------------------------------------------------
# Carga Silver
# ------------------------------------------------------------
def cargar_silver(conn, df: pd.DataFrame) -> int:
    """
    Carga las lecturas limpias de 5 minutos en silver.lectura_5min.

    Usa UPSERT sobre la PK (ts, dispositivo_id): si la lectura ya
    existe se actualiza, si no se inserta. Idempotente.

    Returns:
        Número de filas enviadas.
    """
    sql = """
        INSERT INTO silver.lectura_5min
            (ts, dispositivo_id, p_ac_kw, irradiancia_wm2, temp_modulo_c)
        VALUES %s
        ON CONFLICT (ts, dispositivo_id) DO UPDATE SET
            p_ac_kw         = EXCLUDED.p_ac_kw,
            irradiancia_wm2 = EXCLUDED.irradiancia_wm2,
            temp_modulo_c   = EXCLUDED.temp_modulo_c;
    """
    filas = list(df.itertuples(index=False, name=None))
    with conn.cursor() as cur:
        execute_values(cur, sql, filas)
    conn.commit()
    return len(filas)
```

```python
# ------------------------------------------------------------
# Carga Gold
# ------------------------------------------------------------
def cargar_gold(conn, df: pd.DataFrame) -> int:
    """
    Carga el resumen diario en dwh.fact_energia_dia.

    Usa UPSERT sobre la PK (fecha_key, dispositivo_key). Idempotente.

    Returns:
        Número de filas enviadas.
    """
    sql = """
        INSERT INTO dwh.fact_energia_dia
            (fecha_key, dispositivo_key, fecha, energia_kwh, pct_datos_validos)
        VALUES %s
        ON CONFLICT (fecha_key, dispositivo_key) DO UPDATE SET
            fecha            = EXCLUDED.fecha,
            energia_kwh      = EXCLUDED.energia_kwh,
            pct_datos_validos = EXCLUDED.pct_datos_validos,
            actualizado_en   = NOW();
    """
    filas = list(df.itertuples(index=False, name=None))
    with conn.cursor() as cur:
        execute_values(cur, sql, filas)
    conn.commit()
    return len(filas)
```

Modulo 01_schema.sql (garantiza que una misma lectura no se duplique: si ya existe, se actualizan sus medidas.):

```sql
-- Esquemas medallion
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS dwh;

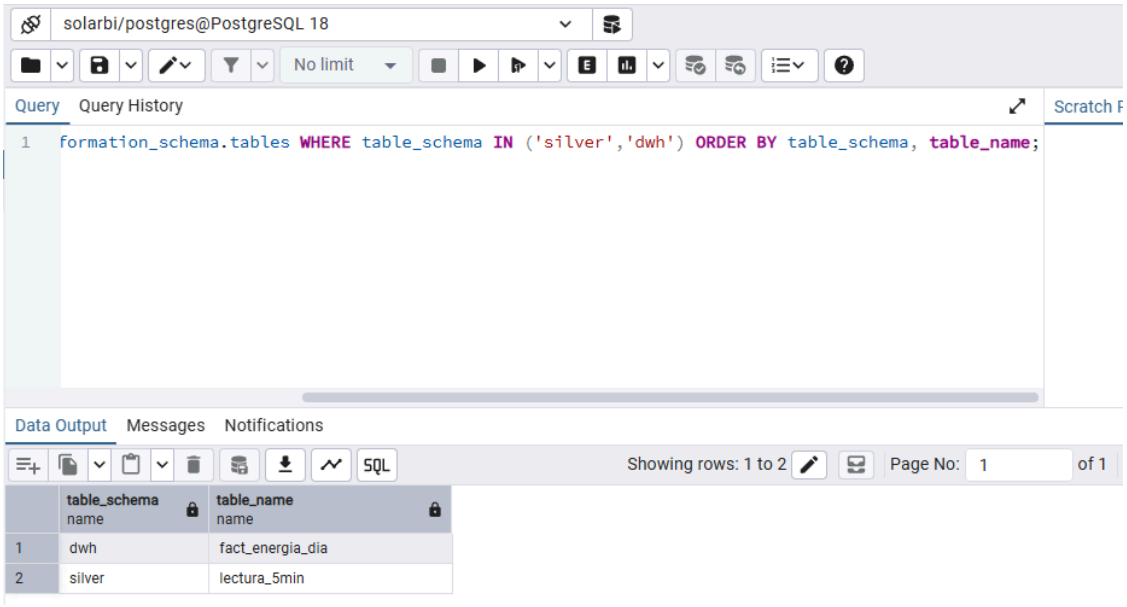
-- Silver: lecturas limpias de 5 minutos
CREATE TABLE IF NOT EXISTS silver.lectura_5min (
    ts              TIMESTAMP     NOT NULL,
    dispositivo_id  INTEGER       NOT NULL,
    p_ac_kw         NUMERIC(6,3)  NOT NULL,
    irradiancia_wm2 NUMERIC(6,1)  NOT NULL,
    temp_modulo_c   NUMERIC(5,1)  NOT NULL,
    PRIMARY KEY (ts, dispositivo_id)
);

-- Gold: resumen diario (esquema estrella simplificado)
CREATE TABLE IF NOT EXISTS dwh.fact_energia_dia (
    fecha_key       INTEGER       NOT NULL,
    dispositivo_key INTEGER       NOT NULL,
    fecha           DATE          NOT NULL,
    energia_kwh     NUMERIC(10,3) NOT NULL,
    pct_datos_validos NUMERIC(5,2) NOT NULL,
    actualizado_en  TIMESTAMP     NOT NULL DEFAULT NOW(),
    PRIMARY KEY (fecha_key, dispositivo_key)
);
```

Estructura en el repositorio

El codigo SQL de creacion de tablas se guarda en sql/01_schema.sql y las consultas UPSERT de referencia en sql/02_upsert.sql. El modulo carga queda en etl/load_postgres.py y el script ejecutable en etl/load_silver.py

Estructura creada en pgadmin:



Evidencia de la carga idempotente

Al ejecutar python -m etl.load_silver dos veces seguidas, el total de filas en silver.lectura_5min permanece constante en 833, lo que demuestra la idempotencia de la carga.

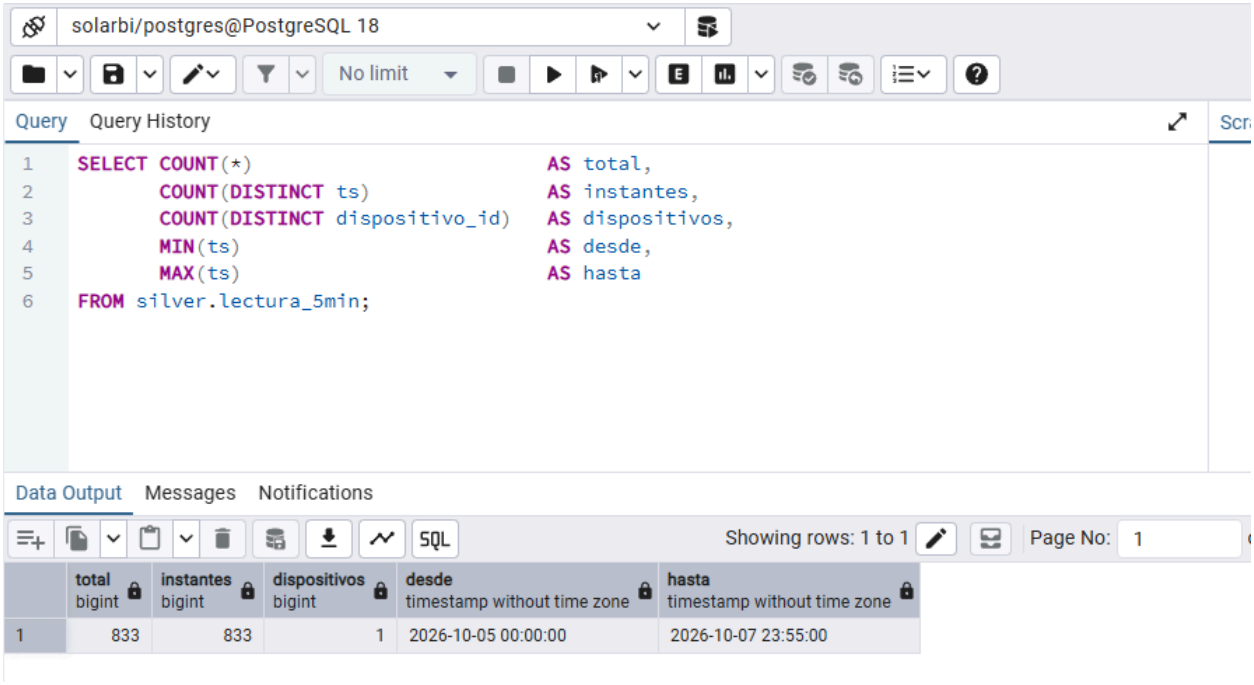
```
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya> python -m etl.load_silver
Leyendo CSV limpio...
Filas en el CSV: 833

Filas enviadas en esta ejecución: 833
Total en silver.lectura_5min    : 833
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya> python -m etl.load_silver
Leyendo CSV limpio...
Filas en el CSV: 833

Filas enviadas en esta ejecución: 833
Total en silver.lectura_5min    : 833
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya>
```

Verificación en PostgreSQL

Desde pgadmin podemos evidenciar que la tabla contiene 833 lecturas, 833 instantes unicos y un unico dispositivo, con cobertura temporal continua del 5 al 7 de octubre de 2026



- **Carguen el resumen diario en una tabla con la estructura de `dwh.fact_energia_dia` (como mínimo: fecha, dispositivo, `energia_kwh` y `pct_datos_validos`). Recuerden que la energía de cada intervalo es la potencia por 5/60 de hora.**

  El resumen diario se construye a partir de las lecturas limpias de 5 minutos aplicando la fórmula energia_kwh = p_ac_kw × 5/60 h para cada intervalo, y luego agregando por día y dispositivo en etl/gold.py. El DataFrame resultante incluye las columnas fecha_key, dispositivo_key, fecha, energia_kwh y pct_datos_validos, tal como exige la tabla dwh.fact_energia_dia. La carga a PostgreSQL se realiza mediante etl/load_gold.py, que ejecuta un UPSERT (INSERT ... ON CONFLICT (fecha_key, dispositivo_key) DO UPDATE). El script se ejecuta con:

```
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya> python -m etl.load_gold
Construyendo resumen diario (Silver -> Gold)...

Resumen a cargar:
 fecha_key  dispositivo_key      fecha  energia_kwh  pct_datos_validos
  20261005               1 2026-10-05       27.223              96.53
  20261006               1 2026-10-06       26.711              95.83
  20261007               1 2026-10-07       26.601              96.88

Filas enviadas en esta ejecución: 3
Total en dwh.fact_energia_dia   : 3
```

Modulo gold.py:

```python
etl > gold.py > construir_fact_diaria
from pathlib import Path
import pandas as pd

RAIZ = Path(__file__).resolve().parent.parent
SILVER = RAIZ / "data" / "silver" / "telemetria_limpia.csv"

# Energía de cada intervalo: potencia (kW) × 5/60 h
INTERVALO_HORAS = 5 / 60


def construir_fact_diaria() -> pd.DataFrame:
    """Lee Silver y agrega la energía diaria por dispositivo."""
    df = pd.read_csv(SILVER, parse_dates=["ts"])
    df["fecha"] = df["ts"].dt.date

    # Energía del intervalo (kWh) y total diario
    df["energia_kwh"] = df["p_ac_kw"] * INTERVALO_HORAS

    agg = (
        df.groupby(["fecha", "dispositivo_id"], as_index=False)
          .agg(
              energia_kwh=("energia_kwh", "sum"),
              lecturas_validas=("ts", "count"),
          )
    )

    # Porcentaje de datos válidos del día (esperado: 288 lecturas = 24h/5min)
    LECTURAS_ESPERADAS = 288
    agg["pct_datos_validos"] = (
        100 * agg["lecturas_validas"] / LECTURAS_ESPERADAS
    ).round(2)

    # Claves del esquema estrella
    agg["fecha_key"] = agg["fecha"].apply(lambda f: int(f.strftime("%Y%m%d")))
    agg["dispositivo_key"] = agg["dispositivo_id"]
    agg["energia_kwh"] = agg["energia_kwh"].round(3)

    return agg[[
        "fecha_key", "dispositivo_key", "fecha",
        "energia_kwh", "pct_datos_validos",
    ]]
```

Evidencia de la carga idempotente:

```
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya> python -m etl.load_gold
Construyendo resumen diario (Silver -> Gold)...

Resumen a cargar:
 fecha_key  dispositivo_key      fecha  energia_kwh  pct_datos_validos
  20261005               1 2026-10-05       27.223              96.53
  20261006               1 2026-10-06       26.711              95.83
  20261007               1 2026-10-07       26.601              96.88

Filas enviadas en esta ejecución: 3
Total en dwh.fact_energia_dia   : 3
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya> python -m etl.load_gold
Construyendo resumen diario (Silver -> Gold)...
Días/dispositivos agregados: 3
```

```
Resumen a cargar:
 fecha_key  dispositivo_key      fecha  energia_kwh  pct_datos_validos
  20261005               1 2026-10-05       27.223              96.53
  20261006               1 2026-10-06       26.711              95.83
  20261007               1 2026-10-07       26.601              96.88

Filas enviadas en esta ejecución: 3
Total en dwh.fact_energia_dia   : 3
PS C:\Users\FAMILIA\Downloads\solarbi-cifuentes-moncada-josue-amaya>
```

Verificación en PostgreSQL:

Query | Query History

```sql
-- Comprobar que la energía diaria ≈ potencia_promedio × horas_de_sol
SELECT fecha,
       energia_kwh,
       pct_datos_validos
FROM dwh.fact_energia_dia
ORDER BY fecha;
```

Data Output | Messages | Notifications

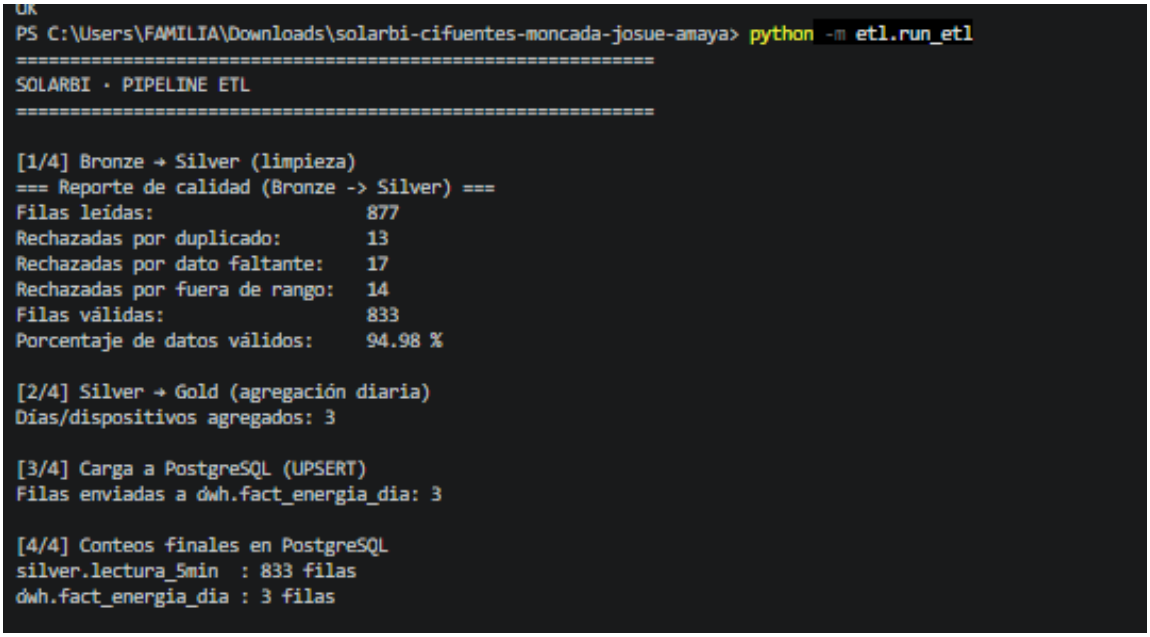
Showing rows: 1 to 3 | Page No: 1

	fecha date	energia_kwh numeric (10,3)	pct_datos_validos numeric (5,2)
1	2026-10-05	27.223	96.53
2	2026-10-06	26.711	95.83
3	2026-10-07	26.601	96.88

El resumen diario se construye a partir de la potencia instantánea (kW) multiplicada por el intervalo de 5 minutos (5/60 h) y se carga de forma idempotente en dwh.fact_energia_dia gracias al UPSERT sobre la clave primaria compuesta (fecha_key, dispositivo_key).

- **Dejen todo en un solo punto de entrada (`etl/run_etl.py`) que se ejecute con un único comando. Ejecútenlo dos veces y demuestren que los conteos no cambian (carga idempotente).**

  El punto de entrada etl/run_etl.py orquesta las tres etapas: limpieza de bronze -> silver con las reglas de calidad, agregación diaria silver -> gold y carga UPSET de ambas capas a PostgreSQL. Se ejecuta con un solo comando desde la raíz del proyecto:



Modulo run_etl.py:

```python
from pathlib import Path
import pandas as pd

from etl.silver import (
    limpiar,
    imprimir_reporte,
    BRONZE,
    SILVER,
    RECHAZADOS,
)
from etl.gold import construir_fact_diaria
from etl.load_postgres import (
    get_conn,
    cargar_silver,
    cargar_gold,
    contar,
)

RAIZ = Path(__file__).resolve().parent.parent


def main() -> None:
    # ================================================
    # 1. Bronze -> Silver: limpieza y reglas de calidad
    # ================================================
    print("=" * 60)
    print("SOLARBI · PIPELINE ETL")
    print("=" * 60)

    print("\n[1/4] Bronze → Silver (limpieza)")
    bronze = pd.read_csv(BRONZE)
    limpio, rechazado, resumen = limpiar(bronze)

    SILVER.parent.mkdir(parents=True, exist_ok=True)
    limpio.to_csv(SILVER, index=False)
    rechazado.to_csv(RECHAZADOS, index=False)
    imprimir_reporte(resumen)

    # ================================================
    # 2. Silver -> Gold: agregación diaria
    # ================================================
```

```python
    print("\n[2/4] Silver → Gold (agregación diaria)")
    fact = construir_fact_diaria()
    print(f"Días/dispositivos agregados: {len(fact)}")

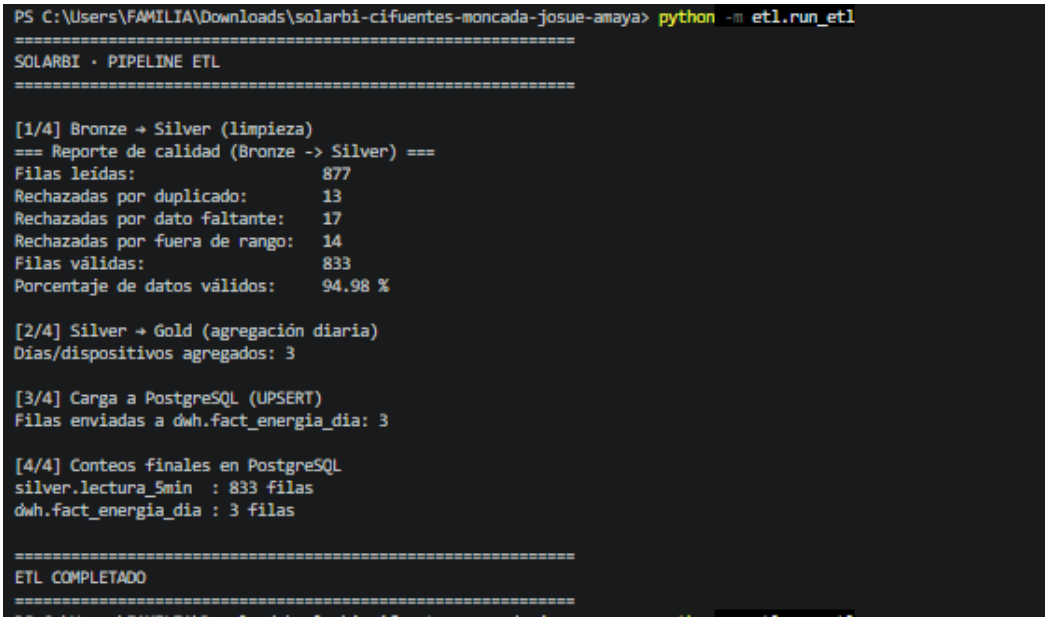
    # ================================================
    # 3. Carga a PostgreSQL: UPSERT idempotente
    # ================================================
    print("\n[3/4] Carga a PostgreSQL (UPSERT)")
    conn = get_conn()
    try:
        n_silver = cargar_silver(conn, limpio)
        n_gold = cargar_gold(conn, fact)
        print(f"Filas enviadas a silver.lectura_5min : {n_silver}")
        print(f"Filas enviadas a dwh.fact_energia_dia: {n_gold}")

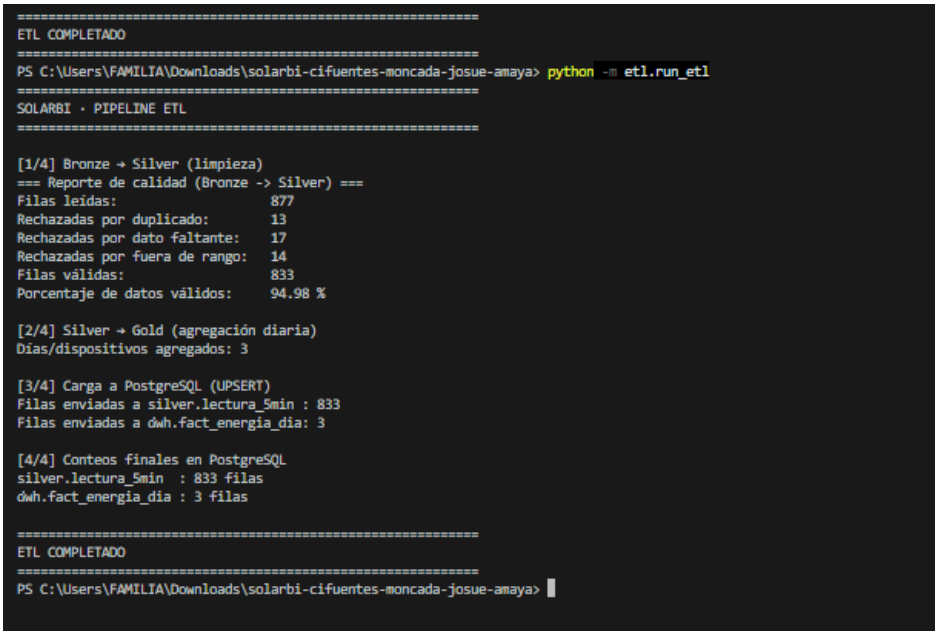
        # ================================================
        # 4. Reporte final: conteos en PostgreSQL
        # ================================================
        print("\n[4/4] Conteos finales en PostgreSQL")
        total_silver = contar(conn, "silver.lectura_5min")
        total_gold = contar(conn, "dwh.fact_energia_dia")
        print(f"silver.lectura_5min  : {total_silver} filas")
        print(f"dwh.fact_energia_dia : {total_gold} filas")

        print("\n" + "=" * 60)
        print("ETL COMPLETADO")
        print("=" * 60)
    finally:
        conn.close()


if __name__ == "__main__":
    main()
```

Evidencia de la idempotencia:





El pipeline completo se ejecuta con un único comando y es idempotente, ejecutarlo dos veces no altera el estado final de PostgreSQL

Verificación:



- **Escriban, sin necesidad de implementarla, la expresión cron (o la configuración del Programador de tareas) que lanzaría ese comando cada día a la medianoche.**

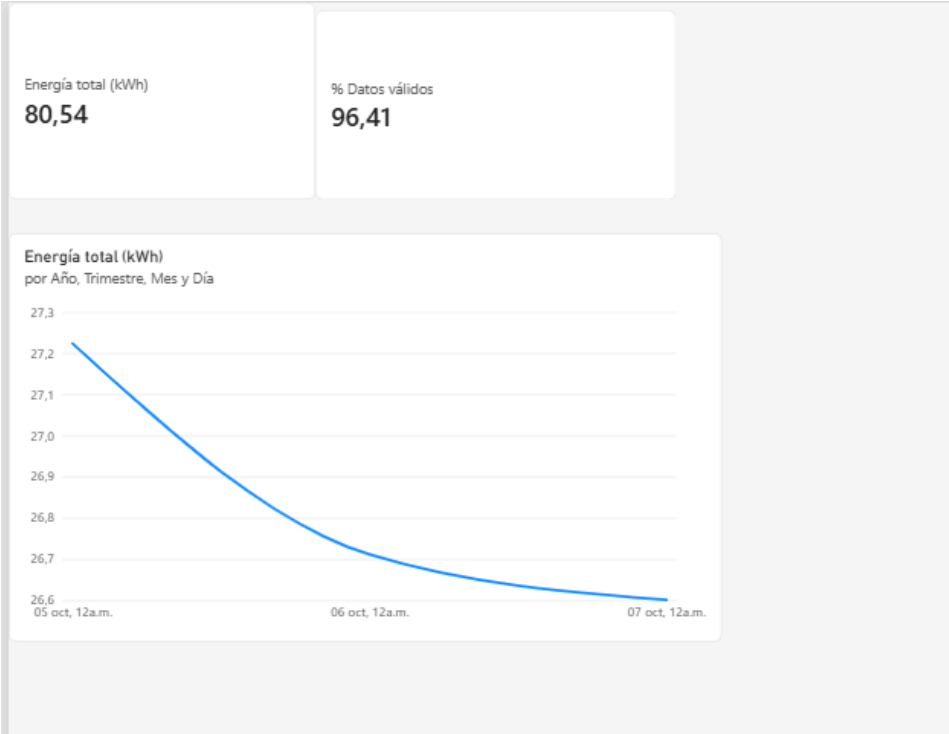
  La expresión se podria implementar en un servidor Linux o macOS se usaria corn con la siguiente expresión:

```
# SolarBI · ETL diario a medianoche

0 0 * * * cd /ruta/al/repo/solarbi-cifuentes-moncada-josue-amaya && /ruta/al/venv/bin/python -m
etl.run_etl >> logs/etl.log 2>&1
```

  La expresión 0 0 * * * indica: minuto 0, hora 0, todos los días del mes, todos los meses, todos los días de la semana.

Paso 4 · Power BI



La página del informe presenta de forma agregada los resultados del pipeline ETL. La primera tarjeta expone la energía total acumulada durante los tres días simulados, alcanzando 80,54 kWh. La segunda tarjeta muestra el porcentaje promedio de datos válidos procesados por las reglas de calidad de la capa Silver, situándose en 96,41 %, lo que indica que la gran mayoría de las lecturas del inversor superaron los controles de duplicados, datos faltantes y rangos físicos. El gráfico de líneas complementa la lectura al mostrar la evolución diaria de la energía generada entre el 5 y el 7 de octubre de 2026, evidenciando una tendencia ligeramente decreciente (27,2 -> 26,9 -> 26,6 kWh). Todas las cifras provienen de medidas DAX y no de columnas arrastradas directamente, en cumplimiento del requisito del taller.

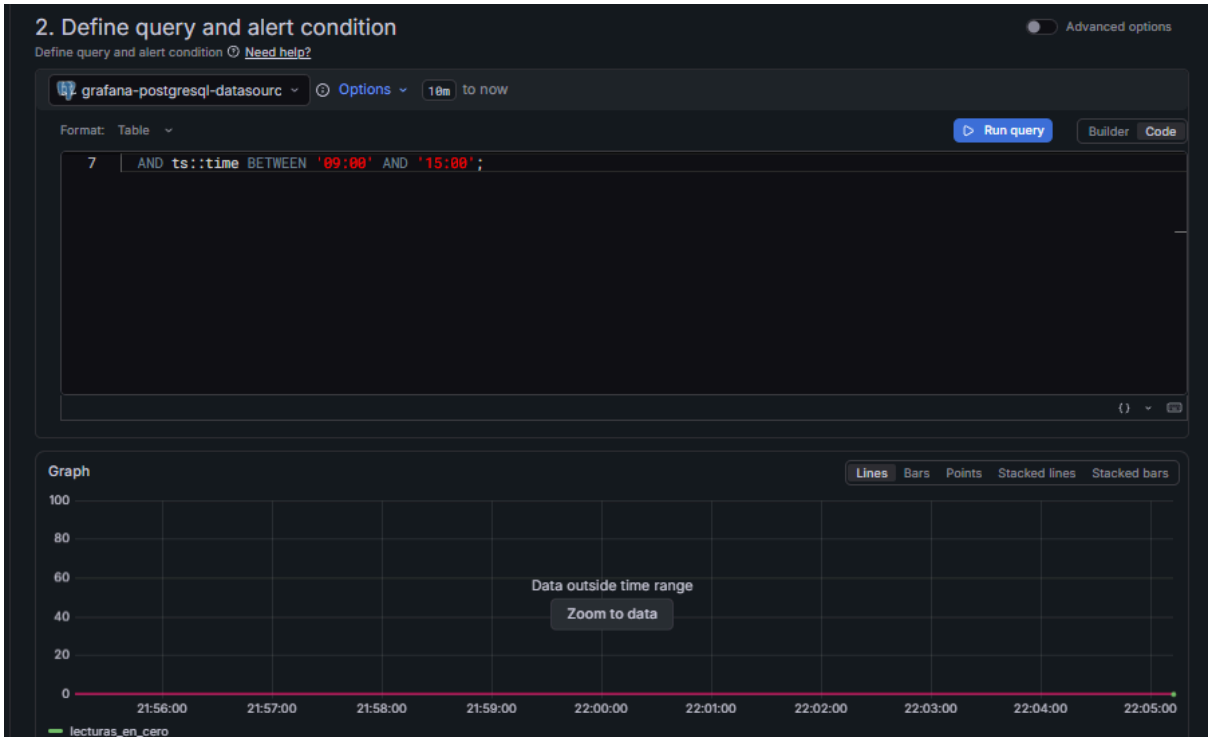
Paso 5 · Grafana

• Un panel de series de tiempo con la potencia p_ac_kw cada 5 minutos, usando $__timeFilter en la consulta.

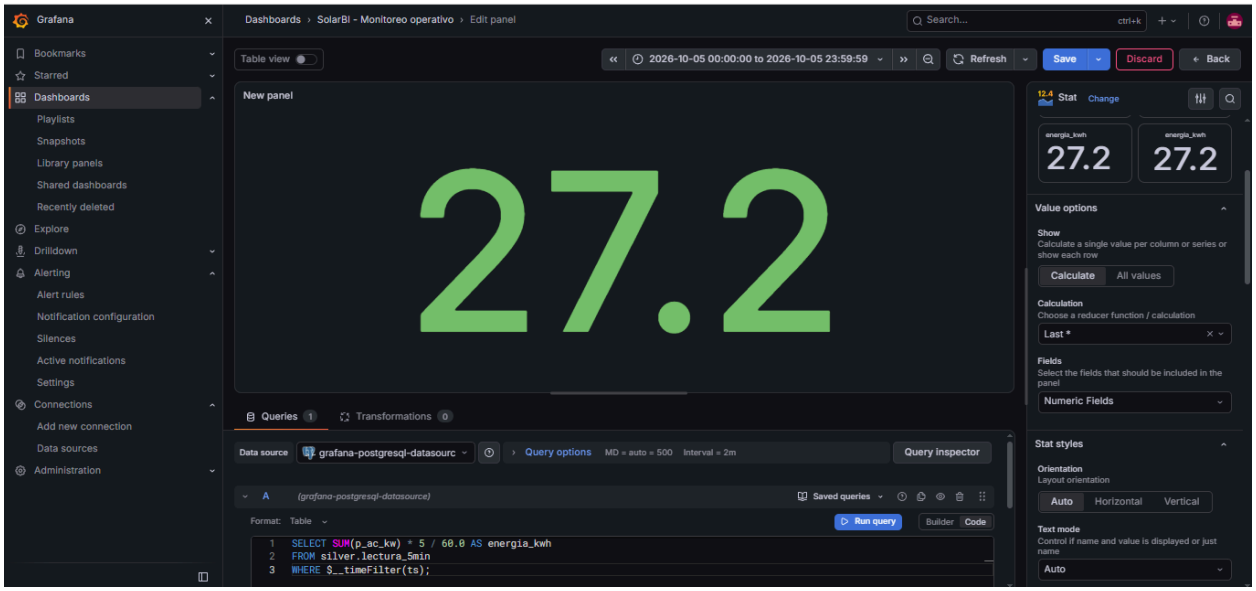


El dashboard de Grafana consulta directamente la tabla silver.lectura_5min de PostgreSQL, la misma base que alimenta Power BI. La consulta usa $__timeGroup para promediar la potencia AC por hora y $__timeFilter para leer solo el rango seleccionado (5 al 7 de octubre de 2026). El panel de series de tiempo muestra el ciclo solar diario, con picos cercanos a 3,8 kW al mediodía y potencia nula en la noche.

- **Un umbral (threshold) o una regla de alerta que se active cuando la potencia sea cero entre las 9:00 y las 15:00.**



- **Un panel tipo stat con la energía del día seleccionado.**



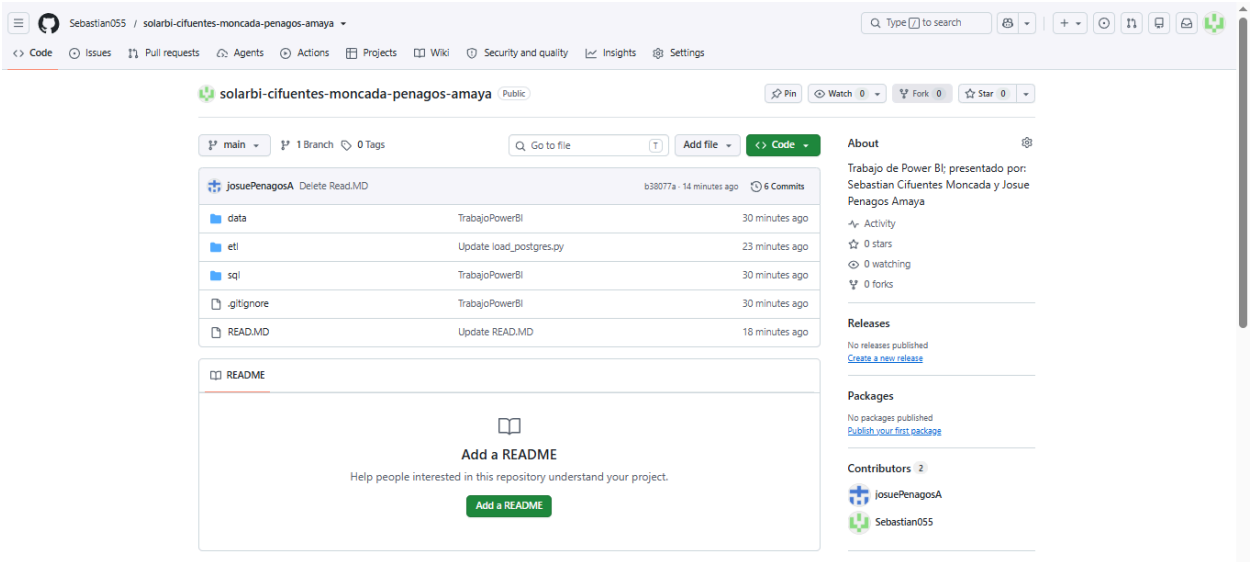
Paso 6 · Reflexión

Con los mismos datos de PostgreSQL, Power BI resolvió mejor el análisis y el resumen: con medidas DAX sobre fact_energia_dia obtuvimos la energía total (80,54 kWh), el % de datos válidos y la tendencia diaria, en un formato claro para quien toma decisiones. Grafana resolvió mejor el monitoreo: con una sola consulta de series de tiempo vimos el ciclo solar hora a hora de las 5 de la mañana a las 6 de la tarde, algo que la granularidad diaria oculta. A Power BI le costó más el modelado, porque las cifras debían salir de medidas DAX y no de columnas arrastradas. En Grafana lo más difícil fue entender las macros $__timeFilter y $__timeGroup y ajustar el rango de tiempo, ya que con “Last 6 hours” el panel salía vacío porque los datos son de octubre. Para el operador de la planta usaríamos Grafana, porque se refresca en segundos y permite alertas continuas. Para un directivo usaríamos Power BI, porque resume resultados y tendencias sin entrar al detalle de cada lectura.

5. Parte C · Repositorio en GitHub

Enlace del repositorio: https://github.com/Sebastian055/solarbi-cifuentes-moncada-penagos-amaya.git

Repositorio:



Colaboradores:

Sebastian055 (Sebastian Cifuentes Moncada)

JosuePenagosA (Josue Penagos Amaya)

Infografía

Bloque 1 · Power BI y Grafana
Puntos: D1–D5
Fuentes:
Data sources | Grafana documentation
Editor de consultas PostgreSQL | Documentación de Grafana
https://grafana.com/docs/grafana/latest/alerting/fundamentals/alert-rules/
documentación de Power BI - Power BI | Microsoft Learn

Bloque 2 · Gobernanza de datos

Puntos: G1–G5

Fuentes:

¿Qué es la gestión de datos? - DAMA International®

DAMA-DMBOK Framework | DAMA DMBOK

Ley 1581 de 2012 - Gestor Normativo - Función Pública

Bloque 3 · Automatización ETL

Puntos: E1–E4

Fuentes:

PostgreSQL: Documentación

Documentación | Apache Airflow

Configurar la actualización programada - Power BI | Microsoft Learn

Planificación de la implementación de Power BI: Auditoría a nivel de datos - Power BI | Microsoft Learn

Bloque 4 · Internet de las Cosas

Puntos: I1–I4

Fuentes:

MQTT Versión 3.1.1

Página de mantenimiento de MQTT | Eclipse Mosquitto

Documentación de Telegraf

PostgreSQL data source | Documentación de Grafana

Bloque 5 · Control de versiones

Puntos: V1

Fuentes:

Git - Referencia

GitHub Docs

Proyectos de escritorio Power BI (PBIP) - Power BI | Microsoft Learn

grafana/docs/sources/as-code/observability-as-code/git-sync/provisioned-dashboards.md en main · Grafana/Grafana · GitHub